run_pipeline.py

Multi-run comparison script: trains every configured (model, feature_set) pair on one shared split

---

A one-command smoke test / experiment-comparison runner. Where `trainer.py` trains and persists exactly one (model, feature_set) combination per invocation, `run_pipeline.py` trains every pair listed in its `RUNS` constant against the *same* loaded sample and time-based split, and prints a single side-by-side comparison table — this is how every cross-feature-set comparison table in this project's development was produced.

RUNS CONST

```
RUNS: list[tuple[str, str]]
```

The list of `(model_name, feature_set)` pairs compared on each invocation. Currently: `logreg` / `hist_gbdt` on `freq_agg`, and `logreg` on each of `baseline_ohe`, `gbdt_leaves`, `gbdt_leaves_ohe`, `gbdt_leaves_concat`, `freq_agg_leaves_concat`, and `freq_agg_fm_concat`. `hist_gbdt` is deliberately not paired with any of the sparse-output feature sets (see `trainer.FEATURE_SET_COMPATIBLE_MODELS`) — it needs dense input, and densifying an uncapped, high-cardinality one-hot matrix would be infeasible.

main FUNCTION

```
python run_pipeline.py [--n-rows] [--val-frac] [--seed] [--data-path]
```

Validates every pair in `RUNS` up front via `trainer.check_feature_set_model_compatible`, loads and time-splits the configured sample once, then trains and evaluates every configured pair against that one shared split — ensuring every row of the printed comparison table is measured on identical data.

Reuses `trainer.engineer_features` / `trainer.build_preprocessor` rather than any local feature-engineering logic, so this script cannot drift out of sync with `trainer.py`'s own single-run behavior. Feature engineering (the expensive part) is cached per `trainer.feature_set_engineering_key`, not per `feature_set` string, avoiding redundant work for feature sets that share identical underlying DataFrame engineering; a *fresh* preprocessor is still built per `(model, feature_set)` run, so no two models ever share a preprocessor instance mutated by a prior `Pipeline.fit`.

Finally prints one row per run: `model/feature_set`, AUC, LogLoss, and PR-AUC (see `evaluator.evaluate` for those metrics' definitions), in a fixed-width text table.